

DOCUSENSE AI

Problem Statement

Contract/Policy Clause Explainer

- Legal and official documents are often difficult to understand.
- Important clauses may contain hidden risks, penalties, or unfavorable conditions.
- Users may struggle with complex legal terminology.
- Most document-analysis tools are primarily English-focused.
- Sensitive documents may not be suitable for uploading to cloud-based AI services.
- Goal: Make complex documents easier to understand, analyze, and question — while keeping processing local and private.

Project: AI Document Intelligence Platform

Interface: Streamlit

AI: Ollama + llama3.2

Embeddings: nomic-embed-text + FAISS

Inputs: PDF, scanned image, raw text

Key Feature: Multilingual explanations and analysis

■ HIGHLIGHTED FEATURE — MULTILINGUAL LANGUAGE TRANSLATION

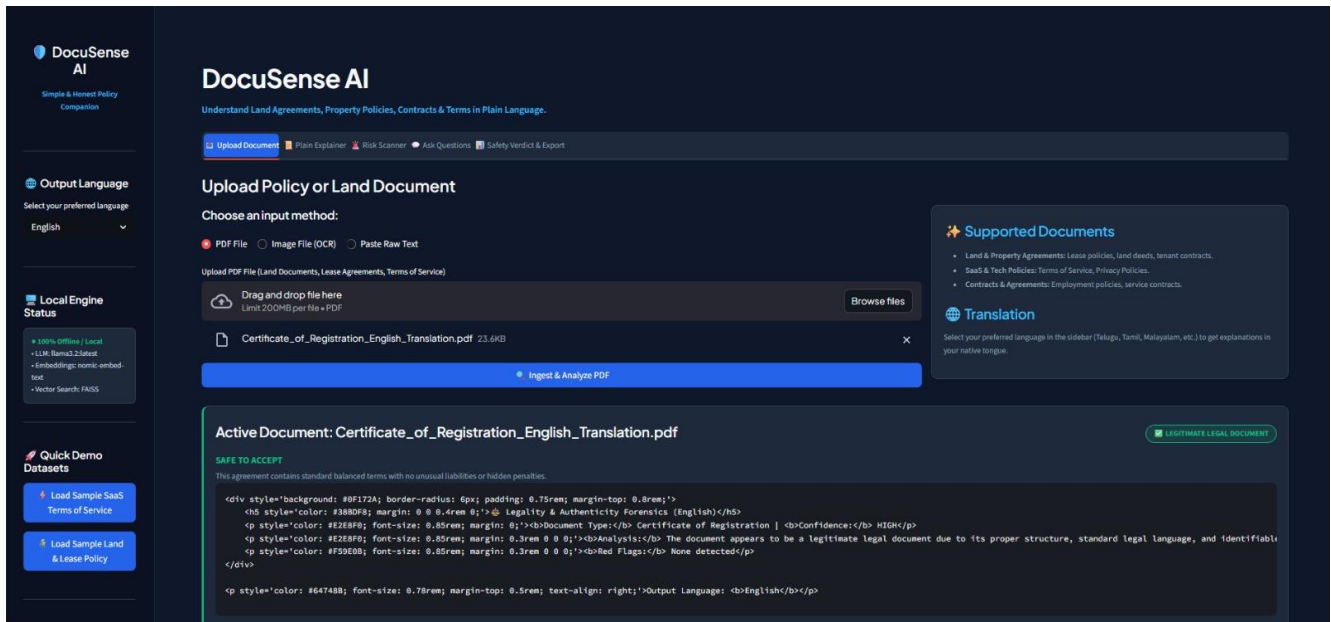
Purpose: The user can select a preferred output language from the sidebar. The current implementation exposes **English, Telugu, Tamil, Malayalam, Hindi, Spanish and French**. The selected **target_language** is passed into clause simplification, risk assessment, document summary and legality/authenticity analysis. This means the AI-generated explanations and analysis are produced in the user's selected language rather than only English.

Executive Overview

DocuSense AI extracts document text, identifies clauses, explains them in plain language, detects potentially risky terms, supports document-grounded Q&A, calculates an explainable safety score, performs legality/authenticity analysis and produces an audit-style report. The application is designed around local AI execution and provides a selectable output language.

Core Architecture

User ☐ Streamlit ☐ PDF/Image/Text ☐ Extraction/OCR ☐ Clause Parser ☐ llama3.2 + Translation ☐ Embeddings ☐ FAISS/RAG ☐ Risk/Summary/Legality ☐ Dashboard ☐ PDF.



Overall System Architecture

The architecture contains the following layers:

```
USER
↓
STREAMLIT WEB UI
↓
PDF / IMAGE / RAW TEXT
↓
PDF EXTRACTION / OCR
↓
CLAUSE PARSER
↓
TARGET LANGUAGE SELECTION
↓
LLM EXPLANATION + RISK ANALYSIS
↓
EMBEDDINGS + FAISS
↓
RAG Q&A; / SUMMARY / LEGALITY
↓
SAFETY SCORE + VERDICT
↓
DASHBOARD + PDF REPORT
```

Technology Stack

Layer	Technology	Purpose
Frontend	Streamlit	Web UI and state management
PDF	pdfplumber / PyPDF	Text extraction
Image	Pillow / EasyOCR	OCR for scanned documents
Parsing	Python Regex	Clause and section detection
LLM	Ollama + llama3.2	Explanation, risk, summary, Q&A
Translation	target_language + LLM prompts	Generate output in selected language
Embedding	nomic-embed-text	Semantic vectors
Retrieval	FAISS	Similarity search
Export	ReportLab	Audit PDF generation

Document Input & Text Extraction

PDF Flow

PDF ☐ pdfplumber ☐ page text ☐ combined text ☐ clause parser. PyPDF acts as a fallback when required.

Image Flow

JPG/JPEG/PNG ☐ Pillow ☐ NumPy/RGB ☐ EasyOCR ☐ extracted text ☐ clause parser.

Raw Text Flow

User pasted text ☐ parse_clauses_from_text(). No PDF or OCR stage is needed.

Language Selection — Important Execution Point

■ HIGHLIGHTED FEATURE — MULTILINGUAL LANGUAGE TRANSLATION

Purpose: The user can select a preferred output language from the sidebar. The current implementation exposes **English, Telugu, Tamil, Malayalam, Hindi, Spanish and French**. The selected **target_language** is passed into clause simplification, risk assessment, document summary and legality/authenticity analysis. This means the AI-generated explanations and analysis are produced in the user's selected language rather than only English.

The application exposes a sidebar control labelled **Output Language**. The available choices in the source code are **English, Telugu, Tamil, Malayalam, Hindi, Spanish and French**. The selected value is stored as **target_language** and passed through the processing pipeline.

Why Translation Is Integrated Into Processing

Translation is not presented as a separate document-conversion stage. Instead, the selected target language is supplied to the AI functions that create explanations, risk analysis, summaries and legality analysis. Therefore the user receives analysis in the selected language while the underlying document text remains available for processing and retrieval.

1. Clause Parsing & Multilingual AI Analysis

Clause Extraction

Regex-based parsing identifies numbered sections and legal headings such as Section, Article and Clause. Each clause is stored with identifiers and text fields. Paragraph chunking acts as a fallback when recognizable headings are insufficient.

Per-Clause Processing

Every extracted clause is passed through two major AI operations: **simplify_clause()** and **assess_clause_risk()**.

Plain-Language Explanation

llama3.2 receives the clause and the selected target language. It creates an understandable explanation while preserving important numbers, names, dates and other relevant facts.

Risk Assessment

The risk scanner combines keyword rules with LLM analysis. Categories include automatic renewal, unilateral modification, binding arbitration, limitation of liability, data exposure, non-refundable payments and heavy penalties.

Risk Levels

LOW • WARNING • CRITICAL

Multilingual Risk Output

The selected target language is also passed to the risk-analysis function. Therefore the reasoning, category explanation and recommended action can be returned in the chosen language.

Embeddings, FAISS & Multilingual RAG Q&A;

Embedding Architecture

Clause title + clause text $\rightarrow$ **numeric-embed-text** $\rightarrow$ **numerical vector** $\rightarrow$ **FAISS**.

The vector store provides semantic retrieval. It is used when the user asks questions about the uploaded document.

RAG Q&A; Flow

User question $\rightarrow$ **question embedding** $\rightarrow$ **FAISS similarity search** $\rightarrow$ **relevant clauses** $\rightarrow$ **llama3.2**
 $\rightarrow$ **grounded answer**.

Multilingual Q&A

The application stores the selected target language in Streamlit session state. Because the same target-language context is used by the AI processing pipeline, the user-facing explanation and analysis can be presented in the selected language.

The source material specifically documents multilingual output for the analysis experience and the sidebar language selector.

Why FAISS Is Needed

For a document with many clauses, the question is converted to a vector and compared against clause vectors. The highest-scoring relevant clauses are retrieved and used as context for the LLM. This is the retrieval component of RAG.

Example User Questions

“What is this document saying?” • “Is it safe to sign?” • “Who is the seller?” • “What are the payment rules?” • “Are there cancellation penalties?”

2. Safety Score, Verdict & Legality Analysis

Document Summary

The system aggregates total clauses, critical risks, warnings, low-risk clauses, safety score, verdict, recommendation and executive summary.

Safety Score

Score = $100 - (\text{Critical} \times 25) - (\text{Warning} \times 10)$, constrained to 10–100.

Verdict Logic

Condition	Result
2+ Critical OR Score < 50	UNSAFE
1 Critical OR 2+ Warnings OR Score < 80	CAUTION
Otherwise	SAFE

Legality & Authenticity Forensics

After summary generation, the full analyzed document is reconstructed and passed to the legality/authenticity analysis function. The result is stored and displayed with the document analysis.

Language-Aware Final Analysis

The source code passes **target_language** into document-summary and legality-analysis functions. Thus the final explanatory summary and legality/authenticity analysis are also designed to respect the

The safety result should be presented as an AI-assisted document risk assessment. The implemented score is a heuristic, and legality analysis is LLM-assisted.

3. Streamlit Dashboard & User Actions

Area	User Action	System Action
Upload	Choose PDF/image/text	Extracts text and starts processing
Language	Select output language	Stores target_language and passes it to AI functions
Plain Explainer	Read clause explanation	Displays original clause + translated/simple explanation
Risk Scanner	Filter critical/warning terms	Displays risk, reasoning and guidance
Ask Questions	Ask free-form question	Retrieves clauses with FAISS and generates answer
Safety Verdict	Review score and verdict	Displays summary, score and recommendations
Export	Generate report	Creates audit-style PDF

Language Translation User Experience

■ HIGHLIGHTED FEATURE — MULTILINGUAL LANGUAGE TRANSLATION

Purpose: The user can select a preferred output language from the sidebar. The current implementation exposes **English, Telugu, Tamil, Malayalam, Hindi, Spanish and French**. The selected **target_language** is passed into clause simplification, risk assessment, document summary and legality/authenticity analysis. This means the AI-generated explanations and analysis are produced in the user's selected language rather than only English.

The source UI explicitly describes translation as a supported feature: the user selects a preferred language in the sidebar and receives explanations in their native tongue. The available language selector contains English, Telugu, Tamil, Malayalam, Hindi, Spanish and French.

What the User Sees

The user can keep the original clause visible while reading the AI explanation and risk guidance in the selected language. This is useful for users who may not be comfortable interpreting technical legal English.

4. Complete Execution Sequence

This sequence incorporates the language-translation action into the application's complete execution path.

#	Action	Execution
1	Open application	Streamlit UI loads.
2	Initialize state	Clauses, vector store, summary, legality, chat and language state are prepared.
3	Select language	User chooses English, Telugu, Tamil, Malayalam, Hindi, Spanish or French.
4	Upload/paste document	PDF, image or raw text is accepted.
5	Extract text	PDF parser or OCR converts input to text.
6	Parse clauses	Regex identifies sections/clauses; fallback chunking may be used.
7	Pass target language	Selected language is supplied to AI analysis functions.
8	Explain clauses	llama3.2 generates plain-language explanations in the selected language.
9	Assess risks	Keyword rules + LLM classify risk and generate guidance in the selected language.
10	Create embeddings	nomic-embed-text converts clause data into vectors.
11	Build FAISS index	Vectors are stored for semantic retrieval.
12	Generate summary	Risk counts, score, recommendation and summary are produced.
13	Legality analysis	Full document is checked; selected language is supplied.
14	Store results	Streamlit session state retains the analysis and target language.
15	Render dashboard	Five functional areas become available.
16	Ask question	User enters a document-related question.
17	Retrieve context	Question embedding is compared with FAISS vectors.
18	Generate answer	llama3.2 produces a grounded answer using retrieved clauses.
19	Review verdict	User inspects safety score, risks and recommendations.
20	Export report	ReportLab generates an audit-style PDF.

Modules, Privacy & Hackathon Presentation

Logical Module Architecture

app.py — main Streamlit application.

pdf_utils.py — PDF extraction, OCR and clause parsing.

llm_utils.py — explanation, risk, summary, Q&A; and legality analysis. **rag_utils.py** — embeddings, FAISS index and similarity search. **pdf_generator.py** — audit PDF generation.

sample_data.py — demo

documents. **theme.py** — UI

styling. **requirements.txt** —

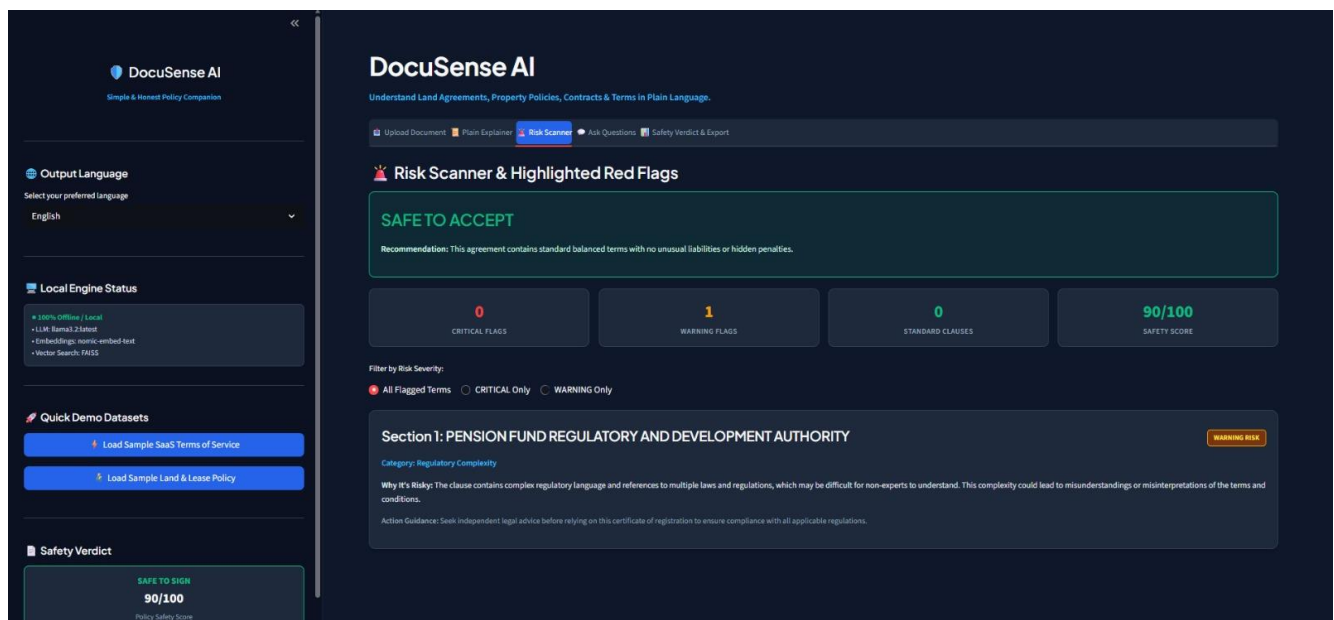
dependencies.

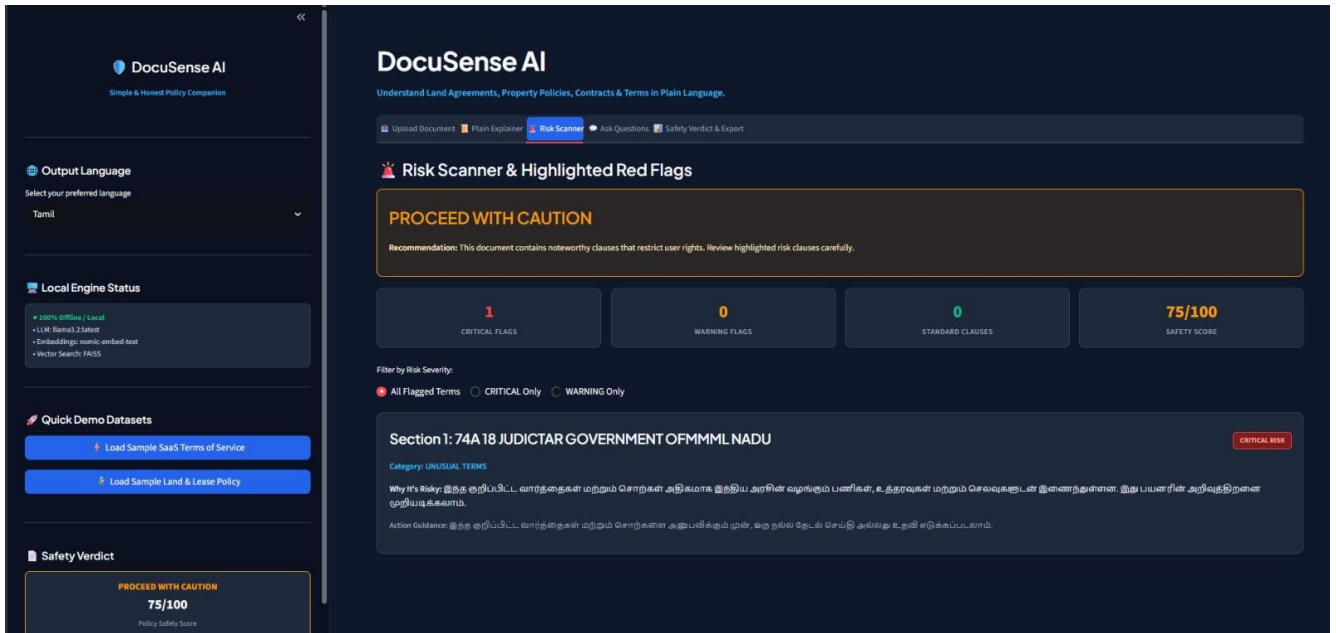
Multilingual Translation — Final Highlight

■ HIGHLIGHTED FEATURE — MULTILINGUAL LANGUAGE TRANSLATION

Purpose: The user can select a preferred output language from the sidebar. The current implementation exposes **English, Telugu, Tamil, Malayalam, Hindi, Spanish and French**. The selected **target_language** is passed into clause simplification, risk assessment, document summary and legality/authenticity analysis. This means the AI-generated explanations and analysis are produced in the user's selected language rather than only English.

The language feature is integrated across the AI pipeline rather than being only a UI label. The selected language is passed into clause simplification, risk assessment, document summary and legality/authenticity analysis. The application therefore supports a multilingual user experience for the core explanatory and analytical outputs





DocuSense AI — Prompt

1. ROLE

You are an expert Python full-stack AI developer, AI/NLP engineer, computer-vision specialist, RAG engineer, Streamlit UI/UX developer, and technical documentation specialist.

You are developing **DocuSense AI**, an AI-powered document analysis application for the **SRM IST Hack & Fest AI Innovation Hackathon — Team 2**.

2. PROJECT CONTEXT

The application is designed for **Contract & Policy Clause Explainer**.

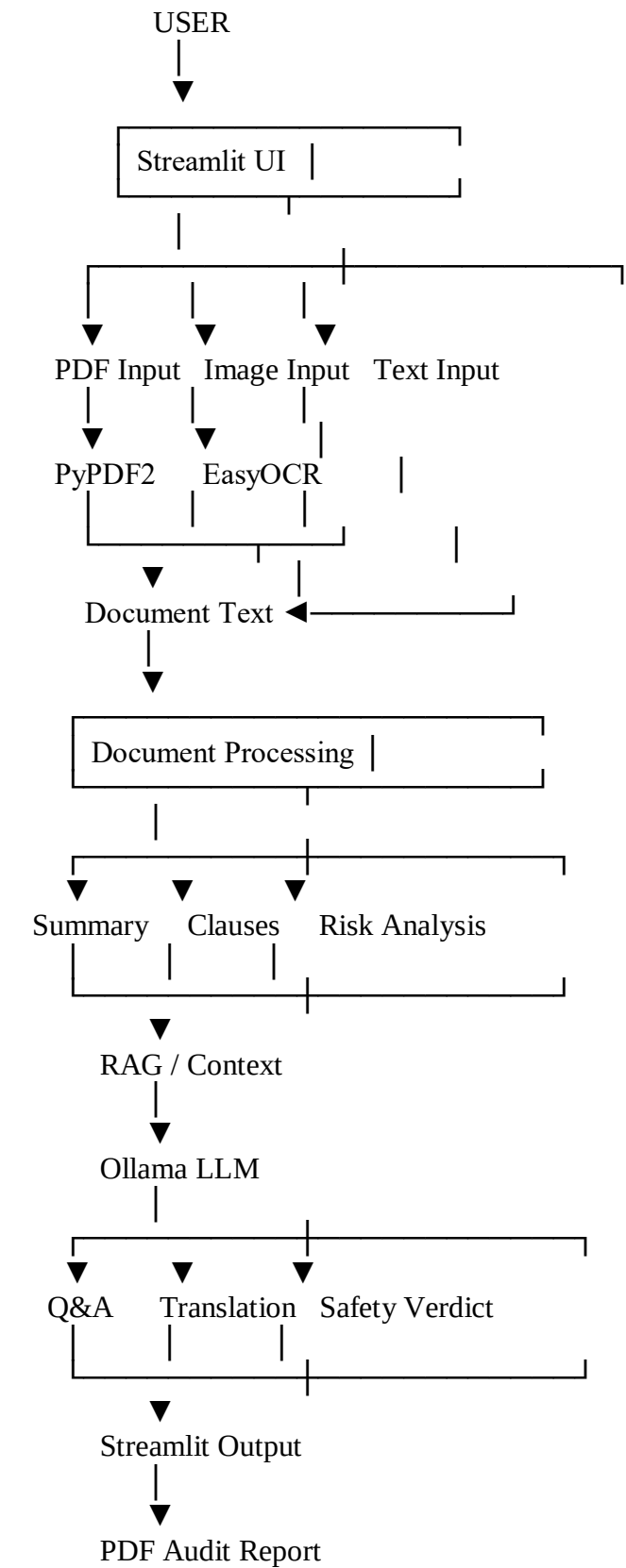
DocuSense AI must allow a user to upload or provide a document and then:

1. Read and process the document.
2. Extract its complete textual content.
3. Identify and analyze clauses.
4. Generate a simple-language summary.
5. Detect unusual, risky, or noteworthy clauses.
6. Calculate a document safety/risk score.
7. Provide a safety verdict.
8. Allow users to ask questions about the document.
9. Answer questions strictly from the uploaded document.
10. Translate generated outputs into selected languages.
11. Support PDF, image, and raw-text inputs.
12. Perform OCR when an image is uploaded.
13. Check whether important legal fields are present.
14. Generate a downloadable professional audit report.

The application must operate **100% locally/offline** using Ollama models. No OpenAI API, Hugging Face API, paid cloud API, or other internet-dependent AI service should be required.

3. TECHNOLOGY ARCHITECTURE

Use the following architecture:



4. PROJECT FILE STRUCTURE

Organize the project into modular Python files:

```
DocuSenseAI/
├── app.py
├── llm_utils.py
├── pdf_utils.py
├── rag_utils.py
├── pdf_generator.py
├── sample_data.py
├── requirements.txt
├── README.md
└── project_prompts_history.txt
```

Module responsibilities

app.py

- Streamlit interface
- File upload
- Input selection
- Language selection
- Tabs
- Q&A interface
- Risk/safety display
- Legality card
- PDF download button

llm_utils.py

- Ollama communication
- Summarization
- Clause explanation
- Risk analysis
- Q&A
- Translation
- Legality analysis
- Strict grounding rules

pdf_utils.py

- PDF text extraction
- Image processing support
- EasyOCR integration
- Unicode-safe processing

rag_utils.py

- Document context preparation
- Retrieval/context management
- Full-context processing for short documents

pdf_generator.py

- Professional PDF report generation
- Unicode font support
- ReportLab formatting

sample_data.py

- Demonstration document
 - Sample South Indian land-sale agreement
-

5. INPUT PROCESSING

The application must support three input methods:

A. PDF Upload

The user uploads a PDF.

Process:

Upload PDF
↓
Read PDF
↓
Extract Text
↓
Validate Extracted Content
↓
Send to Analysis Pipeline

Use **PyPDF2** for PDF extraction.

B. Image Upload

The user uploads an image containing a document.

Process:

Upload Image
↓
Image Processing
↓
EasyOCR
↓
Extract Text
↓
Document Analysis

Replace Tesseract with **EasyOCR**.

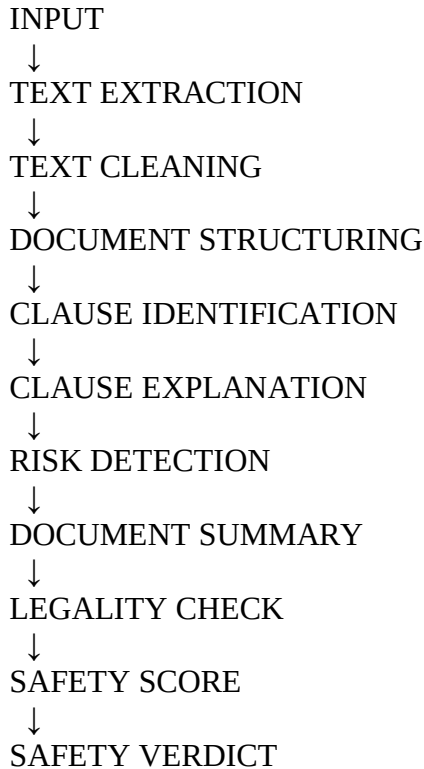
The OCR implementation must avoid Windows Unicode/CP1252 console errors by using UTF-8-safe handling and disabling unnecessary verbose output.

C. Raw Text

Provide a `st.text_area()` allowing the user to paste document text directly.

6. DOCUMENT ANALYSIS PIPELINE

After text extraction, process the document through the following sequence:



The analysis must be based on the uploaded document.

The model must never invent missing facts.

7. CLAUSE ANALYSIS

Identify important clauses such as:

- Parties
- Payment terms
- Ownership
- Responsibilities
- Obligations
- Penalties
- Termination
- Liability
- Confidentiality
- Dispute resolution
- Jurisdiction
- Signatures
- Dates
- Amounts
- Property details

For each important clause, provide:

Clause

↓

Original meaning

↓

Simple-language explanation

↓

Potential concern

↓

Risk level

↓

Reason

8. RISK ANALYSIS

Generate a risk score from **0–100**.

The application should identify potentially concerning clauses and classify them appropriately.

Display:

Policy Safety Score: XX%

Verdict:

SAFE TO SIGN

or

PROCEED WITH CAUTION

or

UNSAFE TO SIGN

Use the risk analysis to determine the displayed verdict. The UI should present the result using professional dashboard components such as metrics, progress indicators, and styled status banners.

9. LEGALITY CHECK

Create:

`check_document_legality()`

The function should inspect the uploaded document for important fields, including:

- Parties
- Signatures
- Dates
- Amounts
- Jurisdiction
- Other relevant mandatory fields supported by the document

Return a structured result:

{


```
"verdict": "...",  
"confidence_score": 0,  
"present_fields": [],  
"missing_fields": [],  
"risk_flags": []  
}
```

Display the result in a dedicated **Legality Check** card.

Important:

The legality assessment must be derived from the uploaded document content rather than fabricated information.

10. RAG AND QUESTION ANSWERING

Create a clearly visible Q&A section.

The user should be able to enter questions such as:

Who is the seller?

What is the total amount?

What is the survey number?

What are the payment terms?

Who is responsible for registration?

Use a visible Streamlit form:

```
graph TD  
    A[Question input] --> B[Ask AI Question]  
    B --> C[Document Context]  
    C --> D[Ollama]  
    D --> E[Grounded Answer]
```

For short documents of approximately **1–10 pages**, use the complete document context where practical instead of relying exclusively on chunk retrieval, because important information such as buyer/seller names can otherwise be split across chunk boundaries.

11. STRICT RAG GROUNDING

This is a mandatory safety rule.

The model must answer **only from the uploaded document**.

If the answer cannot be found in the document, return exactly:

This information is not mentioned in the uploaded document.

Do not:

- Guess
- Use general-world knowledge
- Invent names
- Invent amounts
- Invent dates
- Invent organizations
- Complete missing information
- Hallucinate answers

For example, if the uploaded document is a land deed and the user asks:

Who is the CEO of Google?

The application must not answer from external knowledge.

It must state:

This information is not mentioned in the uploaded document.

This strict grounding requirement is explicitly defined in the project prompts.

12. MULTILINGUAL TRANSLATION

The application must support:

- English
- Tamil
- Telugu
- Malayalam
- Hindi
- Spanish
- French

Add a language selector using a radio button or styled language buttons.

Translation must apply to:

- Document summary
- Clause explanations
- Risk explanations
- Q&A answers
- Other generated explanatory content

The selected language should be applied consistently throughout the application.

13. IMPORTANT TRANSLATION RULE

The following elements must **never be translated or transliterated**:

Names
Currency values
Numbers
Survey numbers
Document IDs
Property identifiers
Reference numbers

Example:

Palanisamy
Muthusamy
₹45,00,000
Survey 123/2A

These must remain in their original form even when the surrounding explanation is translated into Tamil, Telugu, Malayalam, or another language.

This preservation requirement is explicitly specified in the source prompt.

14. SAMPLE DOCUMENT

Create a realistic South Indian land-sale agreement for demonstration purposes.

Use:

Seller: Palanisamy
Buyer: Muthusamy
Amount: ₹45,00,000
Area: 2.50 Acres
Survey Number: 123/2A
Location: Kinathukadavu

The sample must be clearly identified as demonstration/sample content.

Do not treat sample information as real-world verified information.

15. PDF AUDIT REPORT

Add a professional PDF export feature.

Create:

pdf_generator.py

The Streamlit application should provide a download button.

The generated report should contain:

1. Document summary
2. Identified clauses
3. Clause explanations
4. Risk scores
5. Risk flags
6. Safety score
7. Safety verdict
8. Legality check
9. Missing fields
10. Document-based Q&A information where appropriate

The PDF should be readable on Windows without requiring additional font installation.

16. INDIC LANGUAGE PDF SUPPORT

ReportLab's default Helvetica font does not adequately support Indic scripts.

Therefore:

1. Register a Unicode-compatible Windows TTF font.
2. Use the registered font for Indic-language PDF text.
3. Prevent black-square glyphs.
4. Sanitize unsupported emoji characters from PDF title/header sections.

The original project prompt specifically identifies Windows **Segoe UI** TTF registration as the intended approach.

17. PROFESSIONAL UI

Use a professional dark enterprise-style interface.

Recommended visual direction:

Background:

#0F1923

#1A2332

Accent:

Professional gradients

Layout:

Dashboard-style

Clean spacing

Cards

Metrics

Tabs

Progress indicators

Status banners

Avoid a generic bright red/blue/green application appearance.

The main summary page should immediately show:

Document Name
Document Type
Safety Score
Safety Verdict
Risk Level
Legality Status

18. APPLICATION TABS

Use a clear tab structure.

Tab 1 — Overview

Display:

- Document information
- Summary
- Safety score
- Safety verdict
- Legality status

Tab 2 — Clause Analysis

Display:

- Detected clauses
- Simple explanations
- Risk levels
- Risk reasons

Tab 3 — Risk & Legality

Display:

- Risk score
- Risk flags
- Missing fields
- Present fields
- Legality verdict

Tab 4 — Ask AI

Display a prominent question form:

Ask anything about this document

[Enter your question]

[Ask AI Question]

Answers must use document-grounded context only.

Tab 5 — Translation

Allow users to select:

- English
- Tamil
- Telugu
- Malayalam
- Hindi
- Spanish
- French

Display translated outputs while preserving names, numbers, currency, survey numbers, and IDs.

Tab 6 — Export

Provide:

Download Audit Report PDF

19. OLLAMA CONFIGURATION

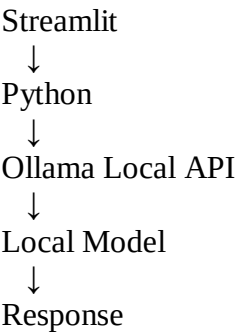
Use local Ollama models:

- llama3.2:latest
- nomic-embed-text:latest

Use the Ollama REST API for local LLM communication.

No external AI API should be required.

The architecture must remain:



20. SAFETY AND GROUNDING SYSTEM PROMPT

Every LLM operation that answers questions about a document must follow these principles:

You are a document analysis assistant.

Use ONLY the supplied document content.

Do not use external knowledge.

Do not guess or fabricate information.

If the requested information is not present in the document, respond exactly:

"This information is not mentioned in the uploaded document."

Preserve names, numbers, currency values, survey numbers, document IDs, and other identifiers exactly as provided.

When translating, translate the explanation but preserve these identifiers in their original form.

21. README DOCUMENTATION

Create a complete GitHub-style README containing:

- Project title
- Problem statement
- Project objective
- Team 2 member table
- System architecture
- Mermaid architecture diagram
- Feature matrix
- Technology stack
- Installation instructions
- Ollama setup
- Model requirements
- Project structure
- Application workflow
- Input-processing workflow
- RAG workflow
- Translation workflow
- OCR workflow
- Risk-analysis workflow
- PDF-export workflow
- Example usage
- Hackathon features

The README should explain the complete application clearly for judges and developers.

22. FINAL EXECUTION FLOW

The completed application must execute approximately as follows:

START



Launch Streamlit



Display DocuSense AI



User selects input type



PDF / Image / Text



Extract document text



Clean and structure text



Analyze document



Identify clauses



Generate summary



Detect risks



Perform legality check



Calculate safety score



Generate safety verdict



Prepare document context



User asks question



Check document context



Generate grounded answer



Translate if requested



Preserve names/numbers/currency/IDs



Display results



Generate PDF audit report



DOWNLOAD



END
